

Simulado 2 de Estrutura de Dados

Nome: _____

- As questões que envolvem implementação devem ser resolvidas utilizando C ou C++.
- Indique claramente o número da questão que está respondendo e, quando necessário, detalhe os passos utilizados para chegar à solução.
- Em questões que envolvem desenhos de árvores, deixe clara a estrutura final obtida.
- Em questões que envolvem simulação de operações, mostre o passo a passo utilizado.

Questão 1 (2.5 pontos) Uma árvore binária é representada pela seguinte estrutura:

```
struct Node {  
    int key;        // Valor armazenado no nó  
    Node* left;     // Ponteiro para o filho à esquerda  
    Node* right;    // Ponteiro para o filho à direita  
};
```

Dizemos que uma árvore binária está balanceada em altura se, para todo nó, a diferença entre as alturas de suas subárvores esquerda e direita é de, no máximo, 1.

Considere que uma árvore vazia é balanceada.

- Implemente a função `int isBalanced(Node* root)`, que retorna 1 caso a árvore esteja balanceada em altura, e 0 caso contrário. É permitido implementar funções auxiliares, se necessário.
- Qual é a complexidade da sua solução no pior caso? Justifique.

Questão 2 (2.5 pontos) Considere o seguinte array:

[12, 7, 25, 3, 18, 30, 5, 20]

Queremos construir uma **min-heap** a partir desse array utilizando a abordagem **bottom-up**, isto é, aplicando `heapify_down` a partir dos últimos nós internos até a raiz.

Considere que o heap é armazenado em array com índices iniciando em 0.

- Mostre o passo a passo das chamadas de `heapify_down`.
- Mostre o array final após a construção da min-heap.

--	--	--	--	--	--	--	--

- Desenhe a representação em árvore binária do heap obtido.

Questão 3 (2.5 pontos) Considere uma árvore AVL inicialmente vazia. Insira, na ordem dada, as seguintes chaves:

30, 20, 10, 25, 40, 50, 22, 24

Sempre que uma rotação for necessária, desenhe a árvore após a rotação e indique:

- qual nó ficou desbalanceado;
- qual foi o caso de desbalanceamento;
- qual rotação ou quais rotações foram realizadas.

Ao final, desenhe a árvore AVL resultante após todas as inserções.

Questão 4 (2.5 pontos) Considere as seguintes chaves a serem inseridas na tabela hash:

10, 20, 30, 40, 21, 31, 41, 51

Em todos os itens abaixo, utilize **encadeamento separado**. Em cada posição da tabela, escreva as chaves armazenadas na ordem em que foram inseridas. Monte a tabela resultante considerando:

- (a) uma tabela de tamanho $m = 13$ e a função hash:

$$h(x) = x \bmod 13.$$

Índice	Chaves armazenadas
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

(b) uma tabela de tamanho $m = 10$ e a função hash:

$$h(x) = x \bmod 10.$$

Índice	Chaves armazenadas
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

(c) Para esse conjunto de chaves, qual das duas funções parece ser melhor? Justifique sua resposta considerando a distribuição das chaves na tabela e a quantidade de colisões.